# CS 391
# Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

**Week - 1**

# Outline

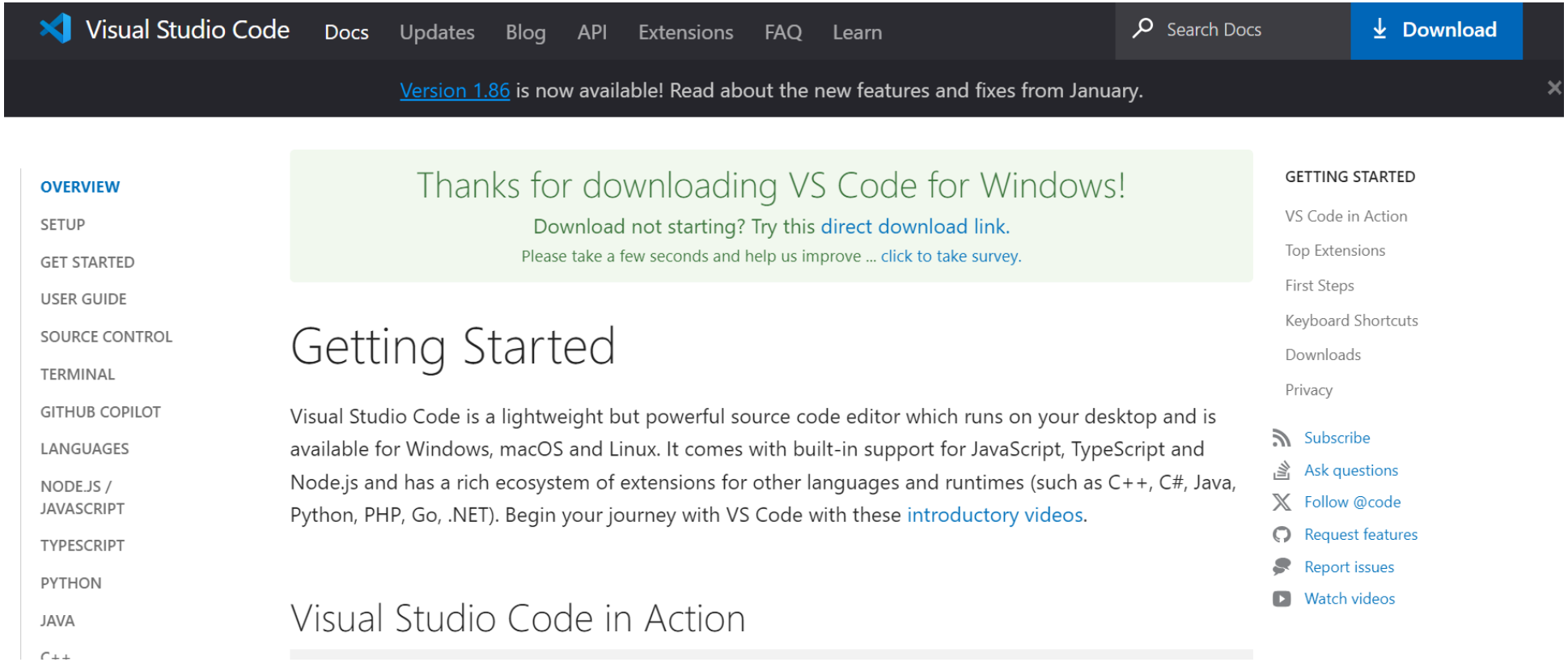
1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# Development Environment

1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# Visual Studio Code

- https://code.visualstudio.com/download



# Postman

- https://www.postman.com/
- API Client
  - Quickly and easily send REST requests directly

# Git

- https://git-scm.com/
- Git is a free and open-source distributed version control system

# Web Architecture Overview

1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# Web Architecture

- The Internet consists of **servers**, **clients**, and **routers**
  - **Clients**
    - A client requests files by sending a HTTP request to server
      - The request is sent over the internet using sockets
      - The file is specified in the request using a Uniform Resource Locator (URL)
    - Access information provided by the servers
    - Web browsers are the most common web clients
  - **Servers**
    - Web servers provide information to web clients
    - When a request comes in from a client, the server “serves” a response
    - The response contains header information as well as the content of the page
  - **Routers** provide the network that allows clients and servers to communicate

# HTTP Request

- The request is formed by the client to inform the server of the request

**http://www.myapp.com:8080/myapp/index.html**

URL: **U**niform **R**esource **L**ocator

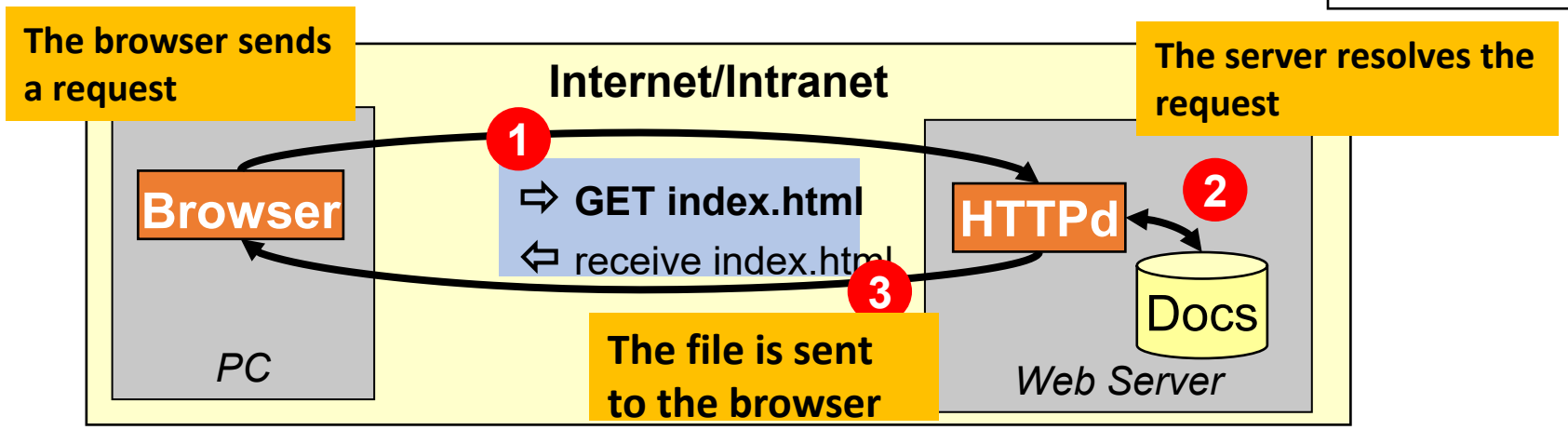
Name of server  Port  Application

name of page to download

```
Get /myapp/index.html HTTP/1.1
Accept: text/html
Accept-Language: en-us
User-Agent: Mozilla
```

**Request Header**

sent to “www.myapp.com:8080”



# HTTP Response

- The server's response includes a header followed by content data
  - The client uses information in the header to determine what to do with the content
- The response header includes
  - Content type, content length, cookies, ...

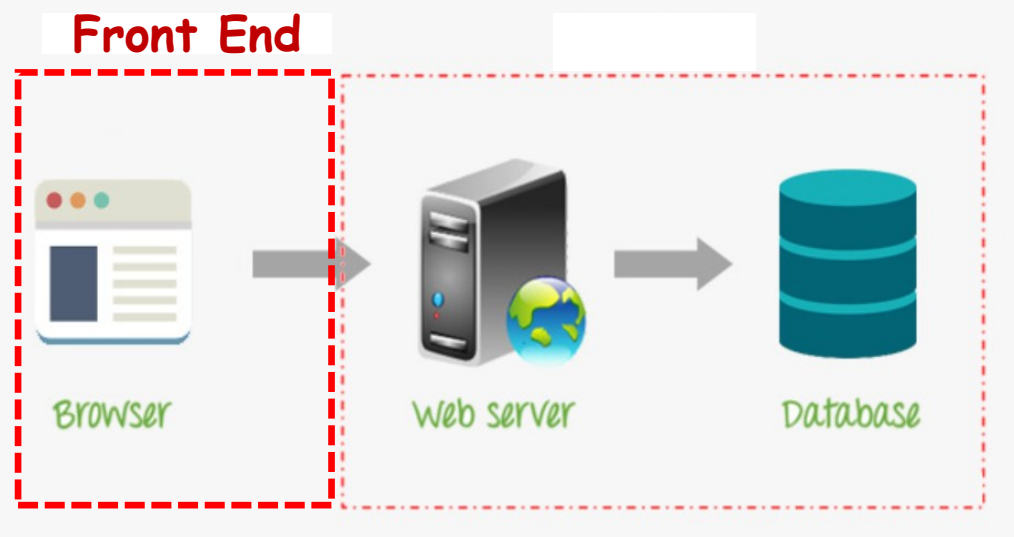
```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2020 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2020 19:15:56 GMT
Content-Length: 88
Content-Type: text/html

<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

index.html

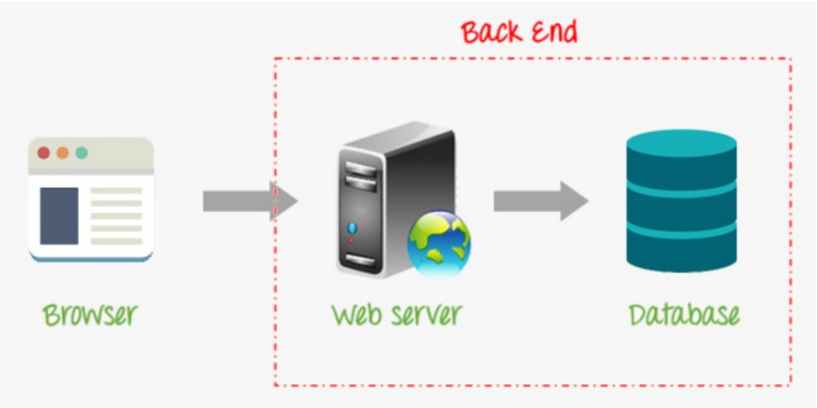
# Front-end Development

- Front-end development manages everything that users visually see first in their browser or application.
- Front-end developers are responsible for the look and feel of a site.
- A front-end developer works with the following technologies:
  - HTML, CSS, and JavaScript
  - Responsive design
  - Bootstrap
  - Frameworks/Libraries: ReactJs, VueJs, Angular, etc



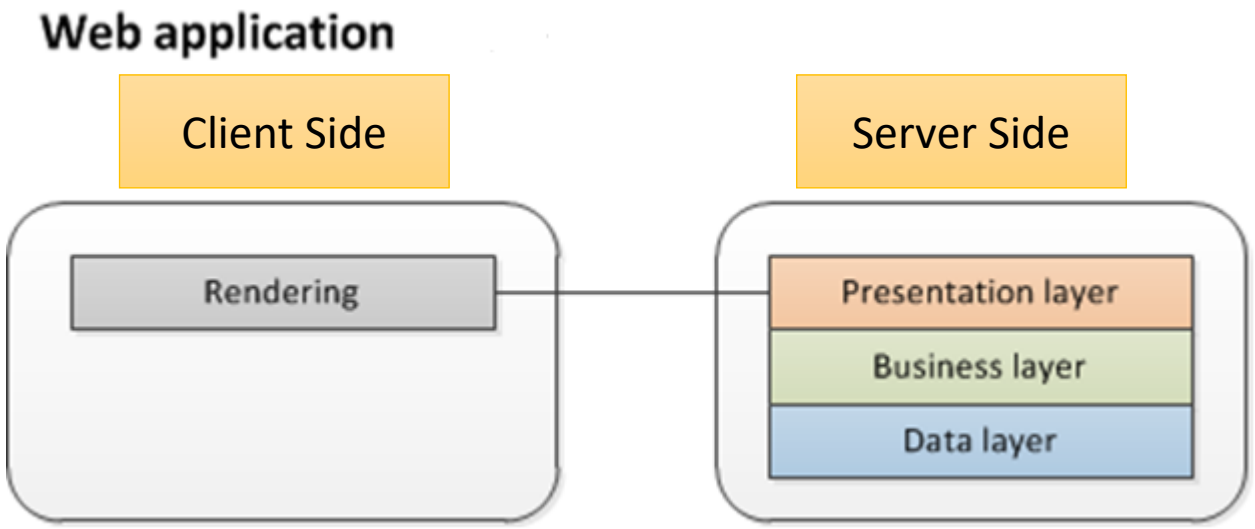
# Back-end Development

- Server-side development is used for the behind-the-scenes activities that happen when performing any action on a website.
- A backend developer works with the following technologies:
  - **Database :** Oracle, DB2, MySQL, MongoDB, SQLServer, etc
  - **Application Server :** Apache, WebLogic, WebSphere, etc
  - Web Services (REST & SOAP)
  - Server-side programming languages like Java, Python, Ruby, . Net, C++, JavaScript, PHP etc.



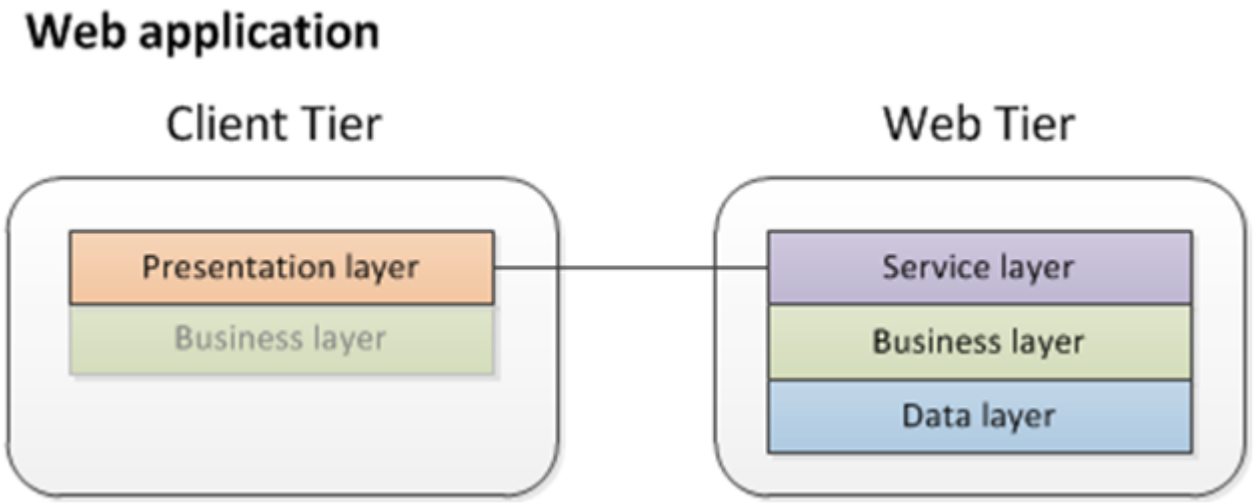
# Presentation Layer Architecture (old style)

- In an “old style” web application, the presentation layer is within the web tier, renders output to the client tier (browser)



# Presentation Layer Architecture (new style)

- Nowadays web sites are turning into Web apps
- Presentation layer moves to client tier
- May also have an element of business logic in client tier



# Presentation Layer Technologies

- Presentation layer technologies are HTML, JavaScript, CSS
- Easy to end up with spaghetti code – global variables, functions whose purpose is unclear, etc.
- Can improve this situation using patterns which have been tried and tested in the presentation layer in the Web Tier
  - Some client-side frameworks/libraries are:
    - Angular
    - React
    - Vue

# Web Standards

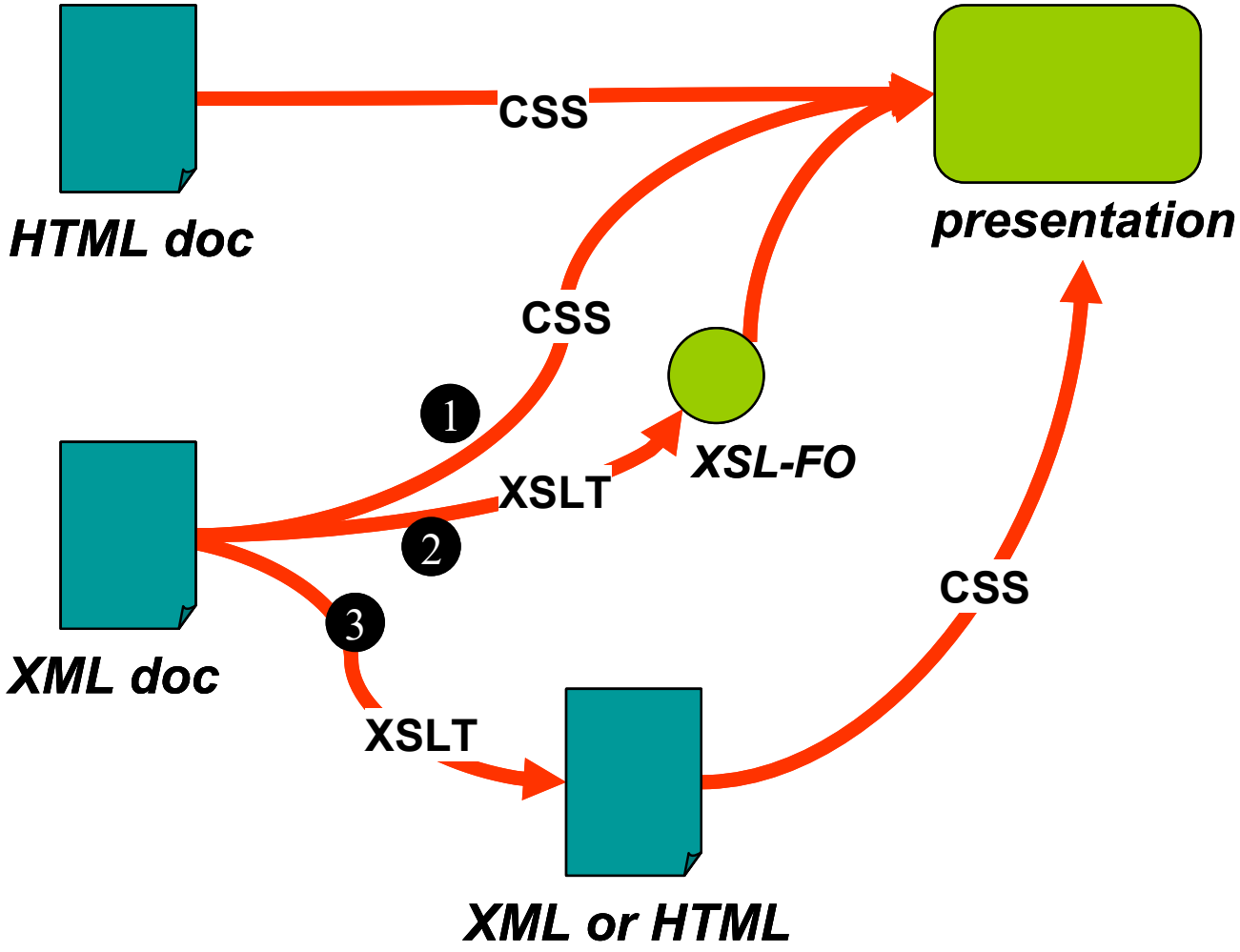
1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# What are Web Standards?

https://www.w3.org/

- Worldwide Web Consortium (W3C)
- Recommends Standards for Web Development
  - Structural Languages
    - HTML – Publishing Language of the Web
    - XHTML - Extensible Hypertext Markup Language
    - XML - Extensible Markup Language
  - Transformations
    - XSL - Extensible Stylesheet Language
    - XPath – XML Path Language
  - Presentation
    - CSS - Cascading Style Sheets

# Standards Related



# Web standards are important

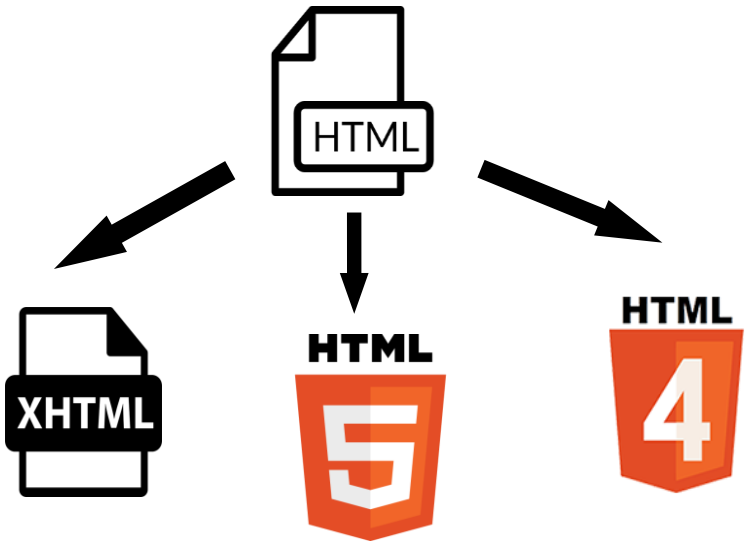
- Designing and building with Web standards
  - Simplicity
    - Simplifies and lowers the cost of production
  - Accessibility
    - Delivers sites that are accessible to more people and more types of Internet devices.
  - Continuity
    - Sites will continue to function correctly  as traditional desktop browsers evolve, and as new Internet devices come to market

# HTML Introduction

1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# HTML Overview

- **HTML: H**yper**T**ext **M**arkup **L**anguage
- HTML files consist of text and tags
  - Text provides the content of the page
  - Tags provide formatting "suggestions" to the client
- HTML tags are **case-insensitive**
  - Lowercase is an HTML5 Convention
- Whitespaces within HTML files are ignored
- HTML elements can be nested
  - start and end tags are properly nested.
    - Ex: text is not allowed.
    - text is allowed.

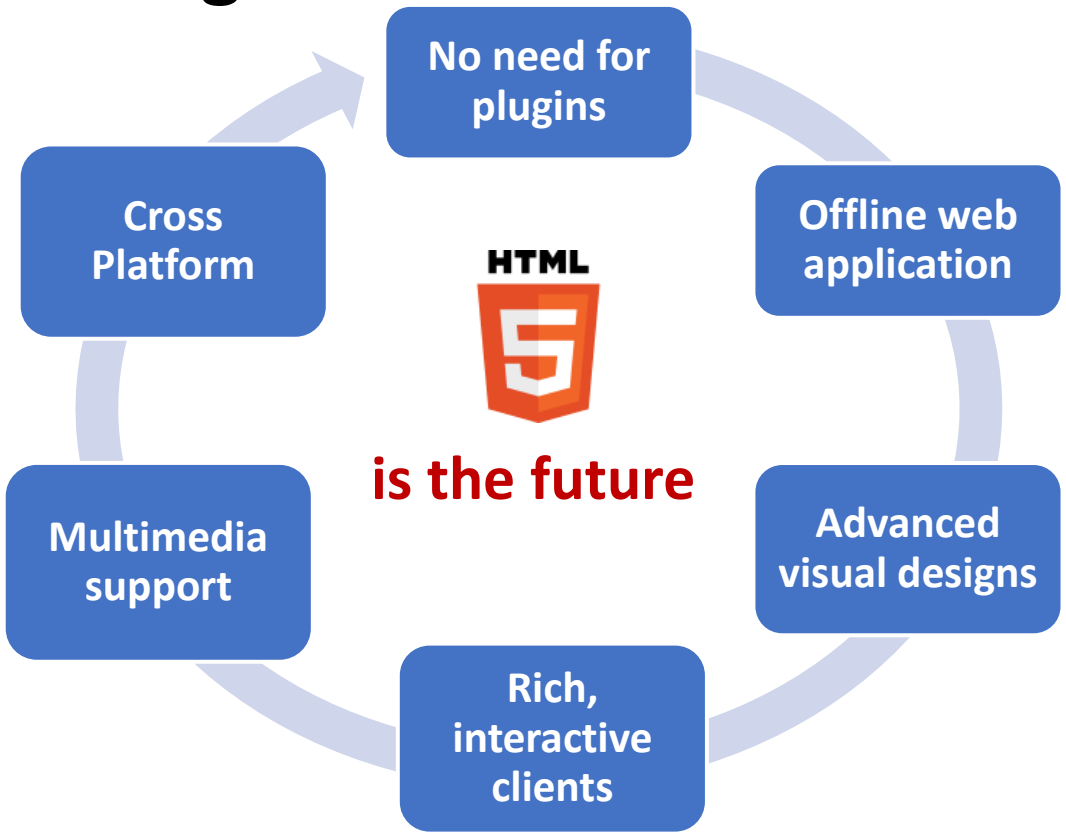


# What is HTML5?

- Latest version of Hypertext Markup Language
  - Currently under development
    - First draft spec at 2008
    - Far away from final version
  - Developed under the HTML 5 working group of the World Wide Web Consortium (W3C)

# HTML5 is not just "one thing"

- HTML5 is a **collection of technologies**
  - Structure by HTML
    - Revised and extended
  - Presentation by CSS
    - Revised and extended
  - Functionality
    - New APIs



# HTML5 Document Structure

<!doctype> declaration

Root element

```
<!DOCTYPE html>
<html>
<head>
      <meta charset="UTF-8">
      <title>Title of the document</title>
</head>
<body>
      …Content of the document…
</body>
</html>
```

# <!DOCTYPE> Declaration…

- All HTML documents must start with a <!DOCTYPE> declaration
  - Not an HTML tag
  - It is an "information" to the browser about what document type to expect.
- The <!DOCTYPE> declaration is NOT case sensitive

# …<!DOCTYPE> Declaration

- HTML 5:
  - <!DOCTYPE html>
- HTML 4.01
  - <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
- XHTML 1.1
  - <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN" "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

# HTML Elements

- An HTML element is defined by a start tag, some content, and an end tag.
  - <tagname>Content goes here...</tagname>
  - <h1>My First Heading</h1>
  - My first paragraph

# Nested HTML Elements

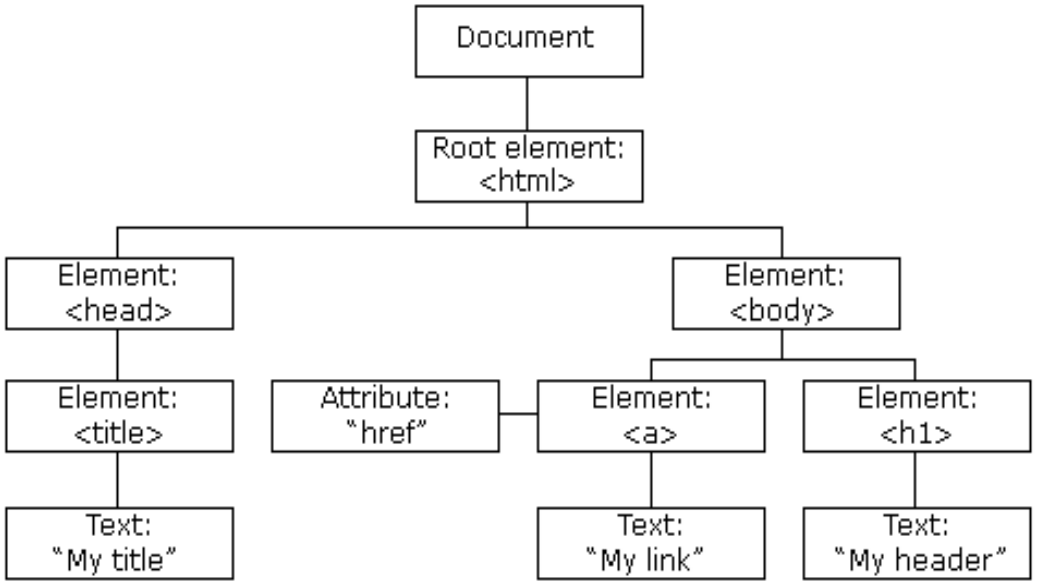
- Elements can contain other elements

```
<html>
  <head>
    <title>Hello html world</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <h1>Introduction to html5</h1>
    <h3>Introduction to <b>tags</b></h3>
    <hr/>
    <a href="https://www.ozyegin.edu.tr/">Ozyegin University</a>
  </body>
</html>
```

# The HTML DOM

- **DOM:** Document Object Model
- Programming interface for HTML
- Defines a standard for :
  - The HTML elements as objects
  - The properties of all HTML elements
  - The methods to access all HTML elements
  - The events for all HTML elements
- When a web page is loaded, the browser creates a Document Object Model of the page.

# DOM Tree




```
<html>
  <head>
    <title>My Title</title>
  </head>

  <body>
    <h1>My header</h1>
    <a href="https://www.mylink.com.tr/">
       My link
    </a>
  </body>

</html>
```

# HTML Attributes

- All HTML elements can have attributes
- Attributes
  - provide additional information about elements
  - always specified in the start tag
  - usually come in name/value pairs like *name="value"*

Element

Start tag

End tag

```
<a href="https://www.ozyegin.edu.tr/" target="_blank">Visit Ozyegin University!</a>
```

attribute value attribute value

# HTML Tags - <meta>...</meta>

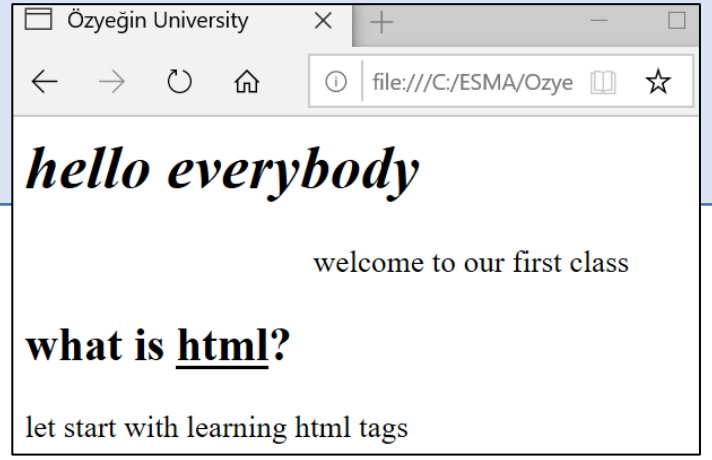
- Metadata is data (information) about data.

```
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Free Web tutorials">
  <meta name="keywords" content="HTML, CSS, JavaScript">
  <meta name="author" content="Erika Felix">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

# Basic Formatting Tags

- <!-- ... --> - Comment
- **<b>** Bold text
- **<i>** Italicized text
- **<u>** Underlined text
- **<br/>** Add a line break to the text
- **<hr/>** Add a line break and header rule
- **<p>** Paragraph
- **<h1>, <h2>, <h3>, <h4>, <h5>, <h6>** headers

```html
<html>
<head>
    <title>Özyeğin University</title>
    <meta charset="UTF-8">
</head>
<!-- This a sample page -->
<body>
    <h1><i>hello everybody</i></h1>
    <p align="center">welcome to our first class
</p>
    <h2> what is <u>html</u>?
    </h2>
    <p>
        let start with learning html tags
    </p>
</body>
</html>
```



# HTML Lists

- HTML has tags that output text in a list format
  - <ul> - Unordered (bullet) list
  - <ol> - Ordered (numbered) list
  - <li> - Start a new entry in a list (ordered or unordered)

```
Shopping list:
<ul>
   <li>Oranges</li>
   <li>Bananas</li>
   <li>Bread</li>
</ul>
```

Shopping list:

- Oranges
- Bananas
- Bread

Shopping list:

1. Oranges
2. Bananas
3. Bread

```
Shopping list:
<ol>
   <li>Oranges</li>
   <li>Bananas</li>
   <li>Bread</li>
</ol>
```

# HTML Links

- Create a hyperlink using the <A> tag
- The most important attribute is the HREF attribute
  - Used to specify the URL of the location to link to
- The link can refer to an HTML page, a servlet, an image, ...

```
<a href="https://www.ozyegin.edu.tr/"> Özyeğin Üniversitesi</a>
```

Özyeğin Üniversitesi

# Table Tags

- A table is specified by providing tags for each row; the columns are specified with each row
- Tags:
  - **<table>** - new table
  - **<tr>** - new row
  - **<td>** - new cell within a row
  - **<th>** - heading cell within a table

```
<table border="1">
    <tr>
        <th>Student</th>
        <th>ID</th>
        <th>Score</th>
    </tr>
    <tr>
         <td>Erika Smith</td>
         <td>12450</td>
         <td>87</td>
    </tr>
    <tr>
         <td>Linda Brown</td>
         <td>34545</td>
         <td>95</td>
    </tr>
</table>
```

| Student | ID | Score |
|---|---|---|
| Erika Smith | 12450 | 87 |
| Linda Brown | 34545 | 95 |

# The Image Tag

- Image tags are used to display graphical images. Some attributes:
- src :
  - source URL of the image
- alt:
  - alternative text to display if the browser can't (or won't) display the graphic
- height , width :
  - Used to customize the size of the image without altering the source file

```
<img src="https://www.ozyegin.edu.tr/sites/all/themes/basic/images/footerlogoTR.png"
    alt="Özyeğin Üniversitesi"
    width="150"
    height="50">
```

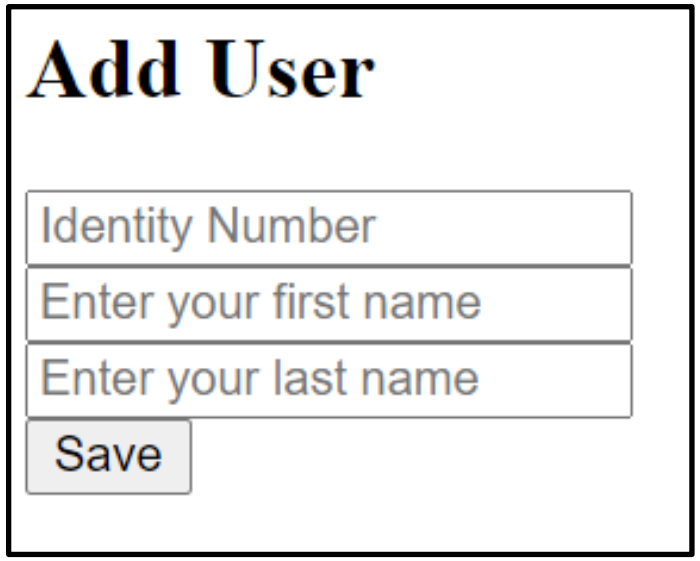
# Html Forms

1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# The Form Tag

- An HTML form is used to collect user input.
- The user input is most often sent to a server for processing.
- Form tags have an **action**
  - Specifies a target URL that is accessed when the form is “submitted”
  - The values for all form elements are packaged and sent to the target
- A web page may contain multiple forms
  - The actions for each form may be the same or different

# Example




```
<h2>Add User</h2>
<form method="POST" action="http://localhost:3000/students">
    <input name="id" type="text" placeholder="Identity Number">
    <input name="firstname" type="text" placeholder="Enter your first name">
    <input name="lastname" type="text" placeholder="Enter your last name">
    <button type="submit">Save</button>
</form>
```

# Form Attributes

- action
  - The target URL that will process the form data
- method
  - How to perform action
  - Determines how the form data is transferred to the target on the server
- target
  - specifies where to display the response that is received after submitting the form
    - _blank : The response is displayed in a new window or tab
    - _self: The response is displayed in the current window (default)
    - _parent : The response is displayed in the parent frame
    - _top The response is displayed in the full body of the window

# Get or Post?

## GET

- This is the default method
- Appends the form data to the URL, in name/value pairs
  - NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

## POST

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked


© Esma Meral

# HTML Form Elements and Attributes

ÖZYEĞİN UNIVERSITY

1. Development Environment
2. Web Architecture Overview
3. What are Web Standards?
4. HTML Introduction
5. HTML Forms
6. HTML Form Elements and Attributes

# Form Elements

- All form elements share certain characteristics
  - The “name” is used to specify the name of the element
  - The “value” specifies the value of the element
    - provides a default value
- The “type” specifies the type of the element
  - e.g., text, checkbox, radio, list, etc.

```
<input type="text" name="firstname" value="Ahmet">
```

# Frequently Used Form Elements

- <input>
- <datalist>
- <label>
- <textarea>
- <button>
- <fieldset>
- <select>
- <option>
- <optgroup>



© Esma Meral

# <input> element

- Most used form element
- Can be displayed in several ways, depending on the type attribute

```
<input type="text" name="name" value="John Smith">
<input type="password" name="pwd">
<input type="submit" value="Submit">
```

# Input Types

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">
- <input type="datetime-local">
- <input type="email">
- <input type="file">
- <input type="hidden">
- <input type="image">
- <input type="month">
- <input type="number">
- <input type="password">
- <input type="radio">
- <input type="range">
- <input type="reset">
- <input type="search">
- <input type="submit">
- <input type="tel">
- <input type="text">
- <input type="time">
- <input type="url">
- <input type="week">



© Esma Meral

# <label> element

- Defines a ***label*** for form elements
- The ***for*** attribute of the <label> tag should be <u>equal to the ***id*** attribute</u> of the <input> element to bind them together.

```
<label for="fname">First name:</label>
<input id="fname" name="firstname" type="text" value="">
```

First name:

# <select> element

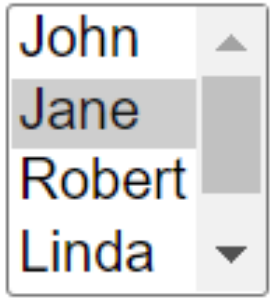
- List permits one or more “options” to be selected
  - The “size” indicates the number of visible lines
  - The “value” for the selected item is provided to your backend

```
<select name="friend" size="4" multiple>
    <option value="1">John</option>
    <option value="2" selected >Jane</option>
    <option value="3">Robert</option>
    <option value="4">Linda</option>
    <option value="5">Erika</option>
</select>
```

select more than one value can be selected

number of visible values

pre-selected



# <textarea> element

- Defines a multi-line input field

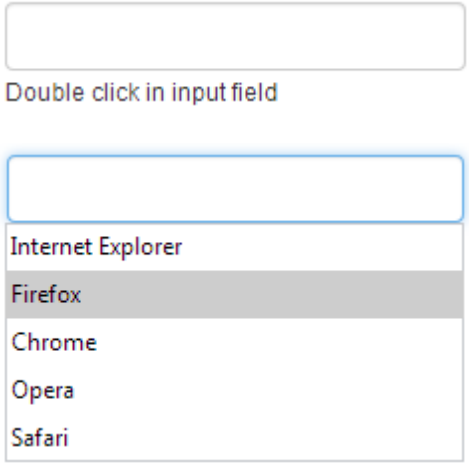
```
<label for="address">Address:</label>
<textarea id= "address" name="address" rows="10" cols="30">
</textarea>
```

Özyeğin University, Çekmeköy Campus
Nişantepe District, Orman Street, 34794 Çekmeköy - İSTANBUL

# <datalist> element

- provide an "autocomplete" feature for an <input> element

```
<form>
   <input list="browsers" name="browser" />
   <datalist id="browsers">
       <option value="Internet Explorer">
       <option value="Firefox">
       <option value="Chrome">
       <option value="Opera">
       <option value="Safari">
   </datalist>
   <input type="submit" />
</form>
```



# <button> element

- defines a clickable button

```
<button type="button" onclick="alert('Hello World!')">Click Me!</button>
```

```
<button type="submit" value="submit">Register Me</button>
```

# Some important input types



© Esma Meral

# <input type="text">

- Defines a single-line text input field:
  - The "value" specifies what is returned

```
<input type="text" name= "city" value="İstanbul">
```

İstanbul

# <input type="checkbox">

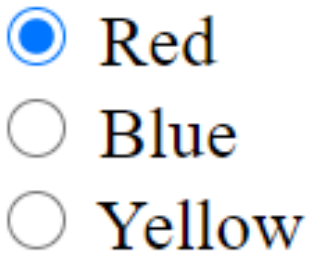
- Checkboxes gather *boolean* information
  - The "value" specifies what is returned if the checkbox is selected
  - The checkbox is initially selected if the checked="true" assigned

```
<label for="3d">I want to pay with 3D Secure</label>
<input id="3d" type="checkbox" name="3dsecure" value="3D Secure" checked="true">
```

I want to pay with 3D Secure ☑

# <input type="radio">

- Defines a radio button
- User can select ONLY ONE of a limited number of choices
- A collection of mutually-exclusive radio buttons share the same “name”
  - The “checked” tag indicates the default value

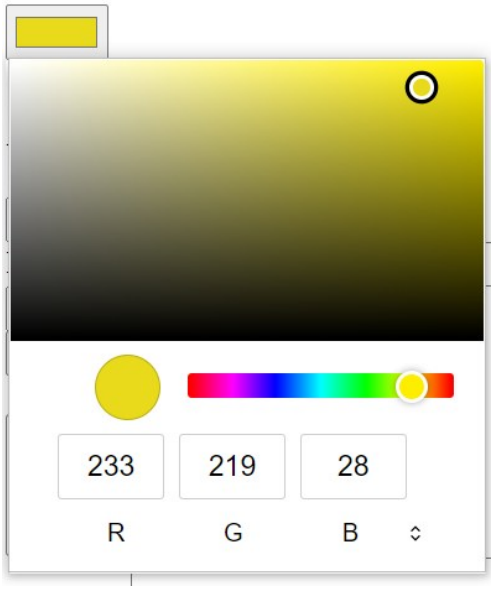



```
<input type="radio" id="red" name="color" value="red" checked="checked">
<label for="red">Red</label><br>
<input type="radio" id="blue" name="color" value="blue">
<label for="female">Blue</label><br>
<input type="radio" id="yellow" name="color" value="yellow">
<label for="yellow">Yellow</label><br>
```

# <input type="color">

- Select a color from a color picker:

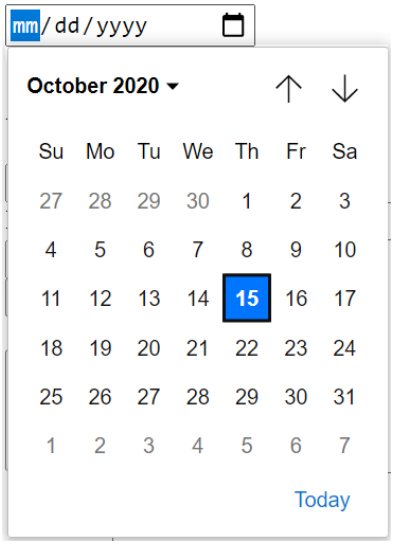
```
<input type="color" name="color" />
```



# <input type="date">
# <input type="time">

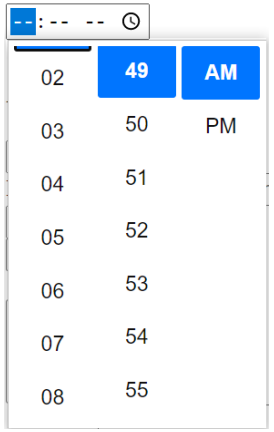
- <date> : allows the user to select a date

```
<input type="date" name="birthday" />
```




- <time> : allows the user to select a time

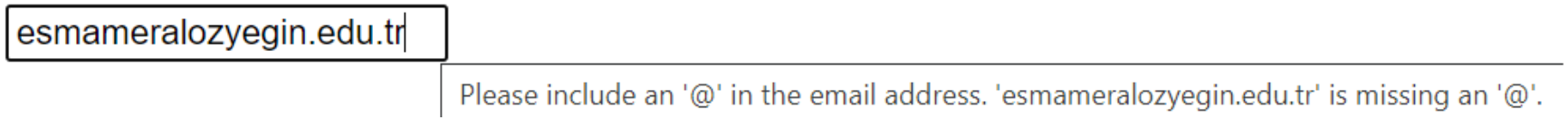
```
<input type="time" name="trxdate" />
```



# <input type="email">

- Used for input fields that should contain an e-mail address.

```
<input type="email" name="email" />
```



# <input type="number">
# <input type="range">

- <number>: input fields that should contain a numeric value

```
<label for="grade">Grade:</label>
<input type="number" id="grade" name="grade" min="1" max="100">
```

Grade: 89

- <range>: defines a control for entering a number whose exact value is not important (like a slider control).

Volume

```
<label for="range">Grade:</label>
<input  id="range" type="range" name="range" min="1" max="10" /> Volume
```

# <input type="submit">

- defines a button for submitting form data

```
<form ...>
    <input name="id" type="text" placeholder="Identity Number">
    <input name="firstname" type="text" placeholder="Enter your first name">
    <input name="lastname" type="text" placeholder="Enter your last name">
    <input type="submit">Submit</button>
</form>
```

Identity Number

Enter your first name

Enter your last name

Submit

# Notes About Submitting Forms…

- The submit event fires when the user clicks a **submit** button or **presses Enter** while editing a field (e.g. <input type="text">) in a form.
  - These 2 items do the same job:

```
<form ...>
   <input type="submit" value="Calculate Age">
   <button type="submit">Find My Age</button>
</form>
```

# …Notes About Submitting Forms

- <input type="submit"> and <button type="submit"> should **be used in forms** only and they will send a request (either GET or POST) to specified URL.
- <input type="button"> can be used anywhere, not just within form and they do not submit form if they are in one.
- The difference is that <button> can have content, whereas <input> cannot
- If you return false from an **onsubmit** event handler, this will prevent the submit of the form to the server from happening

# For More Info and Examples

w3schools.com

https://www.w3schools.com/html
https://www.w3schools.com/html/html_examples.asp

tutorialspoint
SIMPLYEASYLEARNING

https://www.tutorialspoint.com/html/index.htm
https://www.tutorialspoint.com/html5/index.htm

# Any Question?

***Thank you for your attention***

esma.meral@ozyegin.edu.tr



© Esma Meral